Chapitre 3 : Syntaxes de Python

Indentation

- L'indentation fait référence aux espaces au début d'une ligne de code. L'indentation en Python est très importante.
- Python utilise l'indentation pour indiquer un bloc de code.

```
if 5 > 2:
  print("Five is greater than two!")
```

```
[3]: def add(a,b):
         return a+b

     print (add(5,5))
```

```
10
```

Commentaires

- Les commentaires peuvent être utilisés pour expliquer le code Python.
- Les commentaires peuvent rendre le code plus lisible.
- Python n'a pas vraiment de syntaxe pour les commentaires sur plusieurs lignes.
- Cependant, comme Python ignore les chaînes de caractères littérales qui ne sont pas assignées à une variable, vous pouvez ajouter une chaîne multilignes (avec des triples guillemets) dans votre code et placer votre commentaire à l'intérieur.

```
#This is a comment
print("Hello, World!")
```

```
Hello, World!
```

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

```
Hello, World!
```

Variables

- En Python, il n'y a **pas de commande pour déclarer une variable**. Elles sont créées au moment où vous leur assignez une valeur pour la première fois.

- Les variables n'ont **pas besoin d'être déclarées avec un type particulier** et peuvent même changer de type après avoir été définies.

- Cependant, si vous souhaitez spécifier le type de données d'une variable, cela peut être fait par le biais de la **conversion de type**.

```
txt="hello world"
x=5
y=21.3

print(txt)
print(x)
print(y)
```

```
hello world
5
21.3
```

```
x = str(3)    # x will be a string '3'
```

Noms des variables

En Python, un nom de variable :

- **Doit commencer par une lettre ou un caractère de soulignement**
- **Ne peut pas commencer par un chiffre**
- **Ne peut contenir que des caractères alphanumériques et des soulignements (A-z, 0-9, et _)**
- **Est sensible à la casse (age, Age et AGE sont trois variables différentes).**

Types de données intégrés

Python a les types de données suivants intégrés par défaut, dans ces catégories :

- Type texte : str
- Types numériques : int, float, complex
- Types de séquence : list, tuple, range
- Type de mapping : dict
- Types d'ensemble : set, frozenset
- Type booléen : bool
- Types binaires : bytes, memoryview

```
x=2.3
print(type(x))
```

```
<class 'float'>
```

Pour obtenir le type de données de n'importe quel objet, vous pouvez utiliser la fonction type().

Entrées / Sorties

- La fonction input() aide à entrer des données au moment de l'exécution par l'utilisateur, et la fonction print() est utilisée pour afficher le résultat du programme à l'écran après exécution.
- En Python, la fonction input() renvoie toujours une chaîne de caractères (str), quelle que soit la saisie de l'utilisateur. Cela signifie que même si l'utilisateur entre un nombre, il est traité comme une chaîne.

```
nombre = int(input("Entrez un nombre entier : "))
```

- Python 3.6 utilise la méthode input(), tandis que Python 2.7 utilise la méthode raw_input().

```
[5]:
name ="Ibtissame"
print ("Hello "+ name)

Hello Ibtissame
```

```
[7]:
name = input ("Entrez votre prénom :" )
print ("Hello "+name)

Entrez votre prénom : Ibtissame
Hello Ibtissame
```

Les listes

- Les listes sont comme des tableaux à taille dynamique, similaires aux vecteurs en C++ et aux ArrayList en Java.
- Les éléments d'une liste sont ordonnés, modifiables, et peuvent contenir des valeurs en double.
- En Python, une liste est créée en plaçant des éléments entre des crochets [], séparés par des virgules.
- Nous pouvons utiliser l'opérateur d'indexation [] pour accéder à un élément d'une liste, en commençant par l'index 0.
- Python permet l'indexation négative pour ses séquences. L'index -1 fait référence au dernier élément.
- Pour déterminer combien d'éléments une liste contient, utilisez la fonction **len()**

```
MyList =['Python','C++','JavaScript']
print(MyList[0])
```

```
Python
```

```
print(MyList[-1])
```

```
JavaScript
```

```
len(MyList)
```

```
[11]:
3
```

Les opérateurs Arithmétique

+	Additionne deux opérandes ou applique le signe positif unitaire
-	Soustrait l'opérande de droite de celui de gauche ou applique le signe négatif unitaire
*	Multiplie deux opérandes
/	Divise l'opérande de gauche par celui de droite (résultat toujours en flottant)
%	Modulo - reste de la division de l'opérande de gauche par celui de droite
//	Division entière (floor division)
**	Exponentiation - élève l'opérande de gauche à la puissance de celui de droite

Les opérateurs de Comparaison

>	Plus grand que - Renvoie True si l'opérande de gauche est plus grand que celui de droite (x > y)
<	Plus petit que - Renvoie True si l'opérande de gauche est plus petit que celui de droite (x < y)
==	Égal à - Renvoie True si les deux opérandes sont égaux (x == y)
!=	Différent de - Renvoie True si les opérandes ne sont pas égaux (x != y)
>=	Supérieur ou égal à - Renvoie True si l'opérande de gauche est supérieur ou égal à celui de droite (x >= y)
<=	Inférieur ou égal à - Renvoie True si l'opérande de gauche est inférieur ou égal à celui de droite (x <= y)

Les opérateurs de Logique

And	**Renvoie True si les deux opérandes sont vrais**
	(x and y)
or	**Renvoie True si l'un des deux opérandes est vrai**
	(x or y)
not	**Renvoie True si l'opérande est faux (complémente l'opérande)**
	(not x)

Les opérateurs bit à bit

&	Bitwise AND	x & y = 0 (0000 0000)
|	Bitwise OR	x | y = 14 (0000 1110)
~	Bitwise NOT	~x = -11 (1111 0101)
^	Bitwise XOR	x ^ y = 14 (0000 1110)
>>	Bitwise right shift	x >> 2 = 2 (0000 0010)
<<	Bitwise left shift	x << 2 = 40 (0010 1000)

Les opérateurs d'affectation

=	Affectation simple : x = 5
+=	Addition avec affectation : x += 3 (équivalent à x = x + 3)
-=	Soustraction avec affectation : x -= 2 (équivalent à x = x - 2)
*=	Multiplication avec affectation : x *= 4 (équivalent à x = x * 4)
/=	Division avec affectation : x /= 2 (équivalent à x = x / 2)
%=	Modulo avec affectation : x %= 3 (équivalent à x = x % 3)
//=	Division entière avec affectation : x //= 2 (équivalent à x = x // 2)
*=	Exponentiation avec affectation : x **= 3 (équivalent à x = x ** 3)
&=	AND bit à bit avec affectation : x &= y (équivalent à x = x & y)
|=	OR bit à bit avec affectation : x |= y (équivalent à x = x | y)
^=	XOR bit à bit avec affectation : x ^= y (équivalent à x = x ^ y)
<<=	Décalage à gauche avec affectation : x <<= 2 (équivalent à x = x << 2)
>>=	Décalage à droite avec affectation x >>= 2 (équivalent à x = x >> 2)

Opérateurs Spéciaux

Is True	Si les opérandes sont identiques (référencent le même objet)	x is True
is not True	Si les opérandes ne sont pas identiques	x is not True

[13]:

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)
print(a is c)
```

```
True
False
```

Opérateurs d'Appartenance

in	Vrai si la valeur/variable est trouvée dans la séquence	5 in x
not in	Vrai si la valeur/variable n'est pas trouvée dans la séquence	5 not in x

```
fruits = ['pomme', 'banane', 'orange']

print('banane' in fruits)
print('raisin' in fruits)
```

```
True
False
```

Conditions et instructions If

Les instructions conditionnelles sont nécessaires lorsque nous voulons exécuter un code uniquement si une certaine condition est satisfaite.

L'instruction if...elif...else est utilisée en Python pour la prise de décision.

```python
nombre = int(input("Entrez un nombre : "))

if nombre > 0:
    print("Le nombre est positif.")
elif nombre < 0:
    print("Le nombre est négatif.")
else:
    print("Le nombre est zéro.")
```

```
Entrez un nombre :  4
Le nombre est positif.
```

Remarque : Python repose sur l'indentation (espaces au début d'une ligne) pour définir la portée dans le code. D'autres langages de programmation utilisent souvent des accolades à cet effet.

Boucles for

La boucle « for » en Python est utilisée pour itérer sur une séquence (liste, tuple, chaîne) ou d'autres objets itérables. L'itération sur une séquence est appelée traversée.

Le corps de la boucle « for » est séparé du reste du code par une indentation.

```
# Boucle sur une liste de nombres
list = [1, 2, 3, 4, 5]
somme=0
for a in list:
    somme=somme+a
    print(a)

print(somme)
```

Nous pouvons générer une séquence de nombres en utilisant la fonction « range() ». « range(10) » générera des nombres de 0 à 9 (10 nombres).

Boucle while

La boucle « while » en Python est utilisée pour itérer sur un bloc de code tant que l'expression de test (condition) est vraie.
Cette boucle est généralement utilisée lorsque nous ne connaissons pas à l'avance le nombre d'itérations.

```python
# Demande de la limite supérieure
n = int(input("Entrez un nombre entier positif : "))

# Initialisation de la somme et du compteur
somme = 0
i = 1

# Boucle while pour additionner les nombres jusqu'à n
while i <= n:
    somme += i  # Ajoute i à la somme
    i += 1      # Incrémente i pour passer au nombre suivant

print("La somme des nombres naturels jusqu'à", n, "est :", somme)
```

```
Entrez un nombre entier positif :  6
La somme des nombres naturels jusqu'à 6 est : 21
```

Break & Continue

En Python, les instructions **break** et **continue** peuvent modifier le flux normal d'une boucle. L'instruction **break** permet de sortir immédiatement de la boucle dans laquelle elle se trouve. L'instruction **continue** est utilisée pour ignorer le reste du code dans la boucle pour l'itération en cours et passer directement à l'itération suivante. Lorsque continue est rencontré, le contrôle passe à la prochaine itération de la boucle.

```
for var in sequence:
    # codes inside for loop
    if  condition:
        break
    # codes inside for loop

# codes outside for loop
```

```
while test expression:
    # codes inside while loop
    if  condition:
        break
    # codes inside while loop

# codes outside while loop
```

```
for var in sequence:
    # codes inside for loop
    if  condition:
        continue
    # codes inside for loop

# codes outside for loop
```

```
while test expression:
    # codes inside while loop
    if  condition:
        continue
    # codes  inside while loop

# codes outside while loop
```